

Tile flow (what I built)

POST /tiles -> check image -> features from pixels -> local model -> SQLite + PNG on disk



If confidence < 0.50

Still save the row.

Mark needs_review. Don't drop it.

Query later

Filter by class, review flag, or id.

Human can correct the label. Model guess stays.

Notes

- Model is local RandomForest. No cloud vision.
- Train labels = folder names in candidate_tiles. eval_labels.csv is scorecard only.
- I store full scores + a copy of the image, not just the top class.
- SQLite for now. Postgres later if write volume forces it.

Design note

Part 1. Just how I would build this and what I actually shipped.

The idea

A small offline service. You send it a satellite tile, it guesses the land-use class, it saves that guess, and someone can look the results up later. No internet, no OpenAI, nothing in the cloud.

I only built the core path they asked for: take a tile, classify it, store it. I added a bit extra (list, review, a canary check) because otherwise I couldn't tell if the thing was lying to me.

How a tile moves

1. POST a PNG to `/tiles``
2. Check it actually opens as an image
3. Pull some colour / thumbnail features from the pixels
4. Local sklearn model returns a class + confidence + scores for all 7 classes
5. If confidence is under 0.50 I still save the row, I just mark it ``needs_review``
6. Image goes on disk, metadata goes in SQLite
7. GET `/tiles`` or `/tiles/{id}`` to query

That's it. Filename is not used for the label. I learned that the hard way while testing, so I made sure the model only sees pixels.

Pieces

- FastAPI — I know it, one endpoint is easy to explain
- RandomForest (joblib file) — trains on CPU in a minute, no GPU
- SQLite + a folder of PNGs — one process, one box, no Postgres to install

The model is behind a small wrapper. If this was a real GalaxEye machine I would swap in a better model later and leave the API alone.

Choices I actually argued with myself about

What does "query" mean?

I thought about writing a CSV after each run. That's fine for homework, bad if two people want different cuts of the same data. Full map search felt like building a product they said not to build. So I just store rows and filter: class, confidence, review flag, id. Boring, but it answers "show me Forest" and "what's in review".

What do I save?

Not only the top label. I keep the full score vector, model version, filename, and a copy of the image. If something looks wrong next month I want the exact bytes and the exact scores, not a guess. I didn't save the feature vector. That's coupled to this featurizer and I can always recompute it from the PNG.

Unsure predictions

Three options: pretend every guess is fine, drop the weak ones, or keep them and flag them. Dropping them loses data. Pretending is worse. So I keep everything and set ``needs_review`` below 0.50. The number is a bit arbitrary. I would tune it on their eval set if this went further.

Why this model

They said train or use a pretrained local model, and they are not grading accuracy. I trained on the folder names in ``candidate_tiles`` (150 per class). Eval labels I did not train on. I got about 77% on eval. Highway is the messy one. PyTorch felt heavy for this and I wanted something I can change in an interview.

SQLite

Good enough for an offline box. I would move to Postgres if lots of people were writing at once. Same tables.

Stuff I assumed

- Tiles are small RGB PNGs, one class each, like the zip they sent
- Offline means the running app doesn't call the network. pip install once is ok
- They didn't give lat/lon or sensor info so I didn't fake it
- Being wrong a lot is ok if we don't hide it

Questions I would ask

- What decision does the label drive? Forest vs water miss is different from calling a factory a house.
- Do tiles arrive one by one or do we have to cut a big scene?
- Is there already a human in the loop?
- Will the real images still look like EuroSAT or is this a different satellite / resolution?
- How long do we keep the raw tiles?
- Who is allowed to query this?

What I didn't build

Login, a map, a proper GPU model, workers, Postgres. They said stub the rest. I did.

I did add a simple review form and a canary (`POST /canary`) that re-runs the eval set. That's so I can tell if the box is still sane after a month with nobody watching.